Laporan Demo Aplikasi Prototype

MaskGuard: pemantau kepatuhan masker berbasis web dengan komponen AI

Nama	Dito Ramadhan
NIM	2395114035
Program studi	Teknik Informatika
Mata kuliah	Intelligent Mobile and Web Application Development
Tugas	Sesi #7 - Hasil Aplikasi Frontend Web / Prototype
Tanggal	14 Oktober 2026
Repository GitHub	https://github.com/ditormdhn54-sy/maskguard

1. Deskripsi Aplikasi Prototype

Nama aplikasi	MaskGuard
Permasalahan yang diselesaikan	Pemeriksaan kepatuhan penggunaan masker secara manual memakan waktu, bergantung pada ketelitian petugas, dan hasilnya tidak tercatat. MaskGuard mengotomatiskan pemeriksaan dari foto dan menyimpan rekap hasilnya.
Target pengguna	Petugas keamanan dan pengelola fasilitas (kampus, klinik, kantor) yang perlu memantau kepatuhan masker.
Fitur utama	• Unggah foto (klik atau seret) atau ambil foto langsung dari kamera. • Deteksi wajah otomatis, ditandai kotak pada gambar. • Klasifikasi pakai atau tanpa masker untuk setiap wajah, lengkap dengan skor keyakinan. • Ringkasan hasil per foto dan daftar hasil tiap wajah. • Dashboard statistik: jumlah foto, wajah, pemakai masker, dan tingkat kepatuhan. • Riwayat pemindaian tersimpan di database dan dapat dihapus. • Penanganan error (format salah, ukuran lebih dari 5 MB, server tidak terhubung, izin kamera ditolak) dan tampilan kosong (empty state).

2. Arsitektur Sistem

MaskGuard memakai arsitektur klien-server. Browser mengirim foto ke REST API, API meneruskannya ke mesin inferensi, lalu ringkasan hasil disimpan ke database dan dikembalikan ke browser untuk ditampilkan. Frontend dilayani langsung oleh server yang sama sehingga cukup satu proses untuk menjalankan seluruh aplikasi.

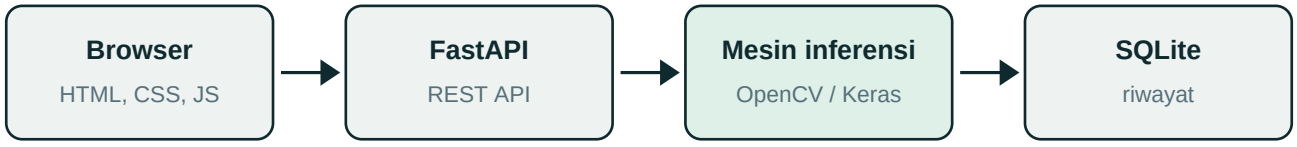


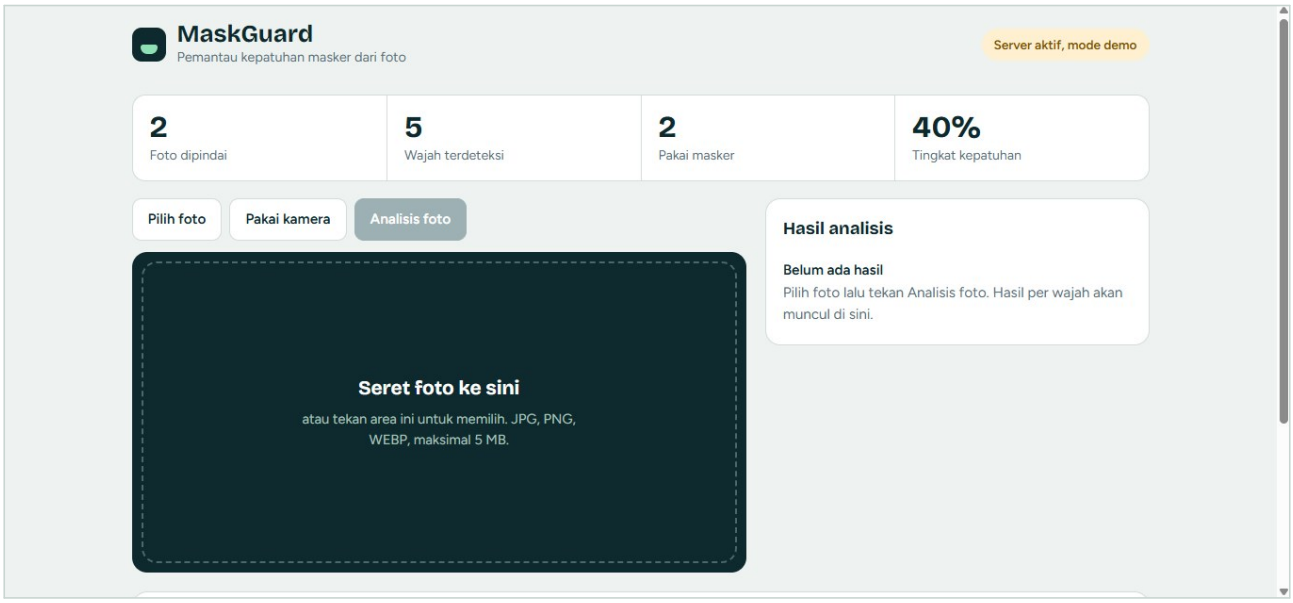
Foto dikirim ke API, dianalisis, lalu ringkasan hasil disimpan dan dikirim kembali ke browser.

Lapisan	Teknologi dan peran
Frontend	HTML5, CSS3, dan JavaScript tanpa framework. Canvas 2D untuk menggambar kotak deteksi, MediaDevices API untuk kamera, Fetch API untuk memanggil backend. Tampilan responsif untuk layar desktop dan ponsel.
Backend	Python 3, FastAPI, dan Uvicorn. Memvalidasi berkas (harus gambar, maksimal 5 MB), memanggil mesin inferensi, dan mengembalikan hasil dalam JSON.
Database / penyimpanan	SQLite (data/app.db), tabel **scans**: id, created_at, filename, faces, mask, no_mask, duration_ms. Hanya ringkasan hasil yang disimpan, gambar tidak disimpan demi privasi.
Integrasi API	REST dengan format JSON. Frontend memakai endpoint pada tabel di bawah.
Komponen AI/ML	Modul backend/inference.py: OpenCV (deteksi wajah Haar Cascade), klasifikasi masker, serta jalur opsional MobileNetV2 (Keras) dan prediksi usia/jenis kelamin (OpenCV DNN). Dijelaskan pada bagian 4.

Metode	Endpoint	Fungsi
GET	/api/health	Status server dan mesin yang aktif
POST	/api/predict	Menerima gambar (field file), mengembalikan hasil per wajah
GET	/api/history	Daftar pemindaian terbaru
GET	/api/stats	Total foto, wajah, pemakai masker, tingkat kepatuhan
DELETE	/api/history	Menghapus seluruh riwayat

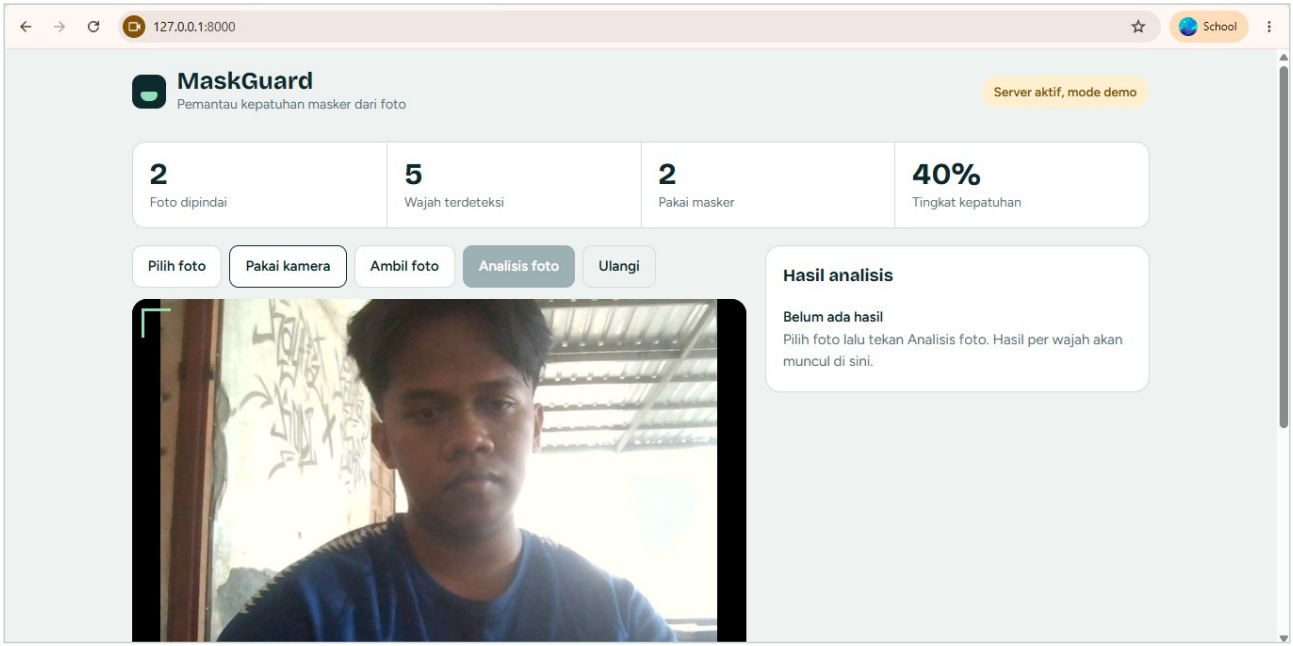
3. Dokumentasi Screenshot

Setiap gambar berikut diambil dari aplikasi yang berjalan di komputer lokal (http://127.0.0.1:8000).



Gambar 1. Halaman utama aplikasi

Tampilan saat aplikasi dibuka. Bagian atas memuat dashboard statistik dan lencana status server (Server aktif, mode demo). Di bawahnya ada tombol aksi dan area unggah foto. Panel Hasil analisis masih kosong (empty state) dan tombol Analisis foto nonaktif karena belum ada foto yang dipilih.



Gambar 2. Proses input pengguna melalui kamera

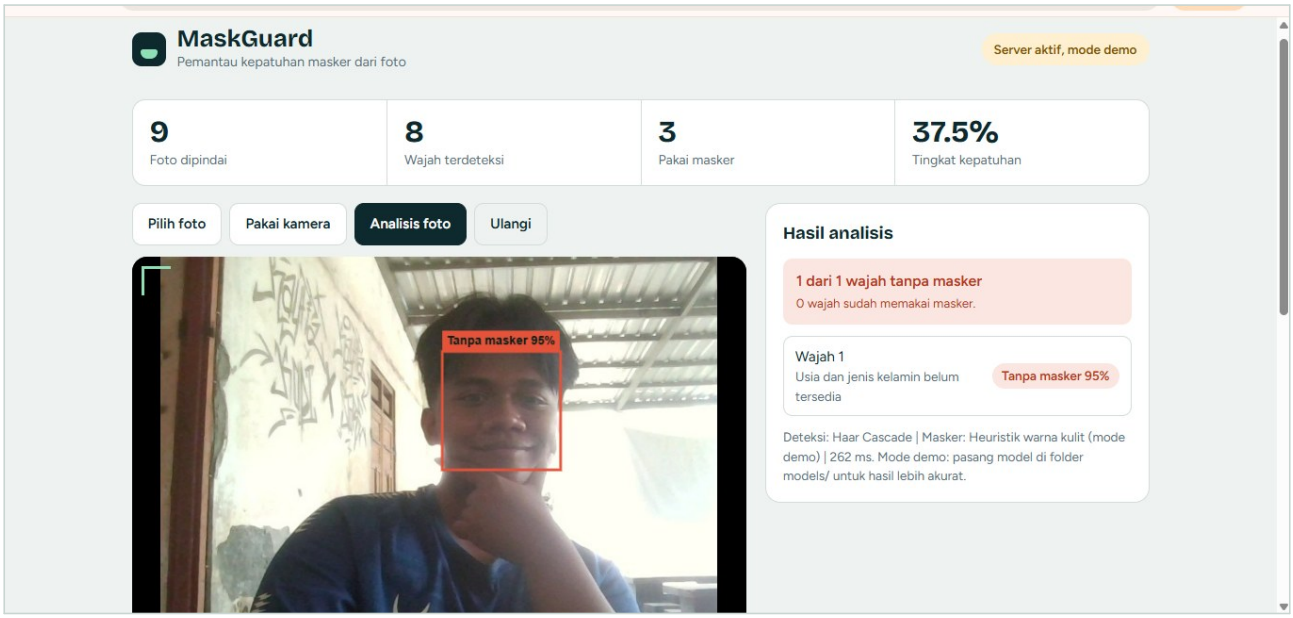
Pengguna menekan Pakai kamera, lalu browser menampilkan pratinjau langsung. Tombol Ambil foto muncul untuk menangkap gambar; setelah itu gambar siap dianalisis. Pengguna juga dapat memilih atau menyeret berkas foto dari perangkat.

TEMPEL SCREENSHOT DI SINI

Klik Analisis foto, lalu segera ambil screenshot saat lapisan gelap Menjalankan inferensi tampil (pakai foto besar agar prosesnya lebih lama).

Gambar 3. Proses loading saat inferensi

Setelah tombol Analisis foto ditekan, lapisan Menjalankan inferensi tampil di atas gambar dan tombol dinonaktifkan sampai server mengembalikan hasil. Ini mencegah permintaan ganda dan memberi tahu pengguna bahwa proses sedang berjalan.



Gambar 4. Hasil prediksi sistem

Wajah ditandai kotak berlabel (contoh: Tanpa masker 95%). Panel Hasil analisis menampilkan ringkasan (1 dari 1 wajah tanpa masker), detail per wajah, mesin yang dipakai, dan waktu proses (262 ms). Dashboard ikut diperbarui: 9 foto, 8 wajah, 3 memakai masker, kepatuhan 37,5%.

TEMPEL SCREENSHOT DI SINI

Pilih berkas PDF (bukan gambar) lewat Pilih foto, lalu screenshot pesan merah yang muncul.

Gambar 5. Tampilan error handling

Pesan berwarna merah muncul bila berkas yang dipilih bukan gambar, ukurannya melebihi 5 MB, atau server tidak terhubung. Pesan menyebut penyebab masalah dan langkah perbaikannya, dan hilang otomatis saat pengguna memilih berkas yang benar.

TEMPEL SCREENSHOT DI SINI

Klik Hapus riwayat, lalu scroll ke bawah dan screenshot bagian Riwayat pemindaian yang kosong.

Gambar 6. Tampilan empty state riwayat

Saat belum ada data, bagian Riwayat pemindaian menampilkan pesan Riwayat masih kosong beserta ajakan mengunggah foto pertama. Kondisi ini muncul pada penggunaan pertama atau setelah riwayat dihapus.

4. Penjelasan Implementasi AI/ML

4.1 Jenis model atau metode yang digunakan

Tahap	Metode	Status pada demo ini
Deteksi wajah	Haar Cascade (Viola-Jones) bawaan OpenCV: haarcascade_frontalface_default, cadangan frontalface_alt2. Model sudah terlatih (pre-trained), tidak dilatih ulang.	Aktif
Klasifikasi masker	Aturan berbasis warna kulit pada ruang warna YCrCb (mode demo).	Aktif

Tahap	Metode	Status pada demo ini
Klasifikasi masker (opsional)	MobileNetV2 hasil transfer learning (Keras), dimuat bila models/mask_detector.h5 tersedia.	Belum dipasang
Usia dan jenis kelamin (opsional)	CNN Levi dan Hassner lewat OpenCV DNN, aktif bila 4 berkas model Caffe tersedia.	Belum dipasang

4.2 Dataset atau sumber data

Prototype ini tidak melakukan pelatihan sehingga tidak memakai dataset pelatihan. Data masukan adalah foto yang diunggah atau diambil dari kamera saat aplikasi dipakai, dan foto tidak disimpan. Model Haar Cascade berasal dari paket OpenCV yang sudah dilatih oleh pengembang OpenCV. Untuk jalur MobileNetV2, dataset citra wajah bermasker dan tanpa masker (misalnya dari Kaggle) akan dipakai pada tahap pengembangan berikutnya.

4.3 Tahapan preprocessing data

- Dekode gambar, koreksi orientasi EXIF (foto dari ponsel), dan konversi ke RGB.
- Pengecilan gambar untuk deteksi: sisi terpanjang maksimal 1000 px. Koordinat hasil dikembalikan ke ukuran asli.
- Konversi ke grayscale dan equalizeHist untuk meratakan pencahayaan.
- Deteksi multi-skala (scaleFactor 1,1; minNeighbors 5; ukuran wajah minimum 6% dari sisi terpendek gambar).
- Setiap wajah dipotong (crop). Untuk klasifikasi aturan, area atas wajah (dahi dan mata) dan area bawah (hidung dan mulut) dikonversi ke YCrCb. Untuk MobileNetV2: resize 224x224 dan normalisasi piksel ke rentang -1 sampai 1.

4.4 Proses training atau penggunaan model

Tidak ada proses training pada prototype ini. Seluruh komponen dipakai dalam mode inferensi saja. Mesin dimuat satu kali saat server dinyalakan agar permintaan berikutnya cepat. Jalur pengembangan: melatih MobileNetV2 (bobot awal ImageNet, kepala klasifikasi 2 kelas) lalu menyimpannya sebagai mask_detector.h5. Server akan memakainya otomatis tanpa perubahan pada antarmuka.

4.5 Cara model menghasilkan output

Untuk setiap wajah, mesin menghitung proporsi piksel kulit pada area atas (u) dan area bawah (l), lalu rasio r = l / u. Jika r kurang dari 0,45 maka area mulut dan hidung tidak berwarna kulit sehingga wajah dinilai memakai masker; selain itu dinilai tanpa masker. Skor keyakinan (0,50 sampai 0,95) dihitung dari seberapa jauh r dari ambang tersebut. Skor ini adalah skor heuristik, bukan probabilitas dari model terlatih. Hasil dikirim sebagai JSON, contohnya:

```
{ "width": 640, "height": 480, "duration_ms": 262,
  "faces": [ { "box": {"x":176,"y":66,"w":95,"h":95},
    "has_mask": false, "label": "Tanpa masker",
    "confidence": 0.949, "age": null, "gender": null } ],
  "summary": { "total": 1, "mask": 0, "no_mask": 1 } }
```

4.6 Evaluasi dan pengukuran performa

Evaluasi kuantitatif (accuracy, precision, recall) belum dilakukan karena belum tersedia dataset uji berlabel. Yang dilakukan adalah pengujian fungsional dan pengamatan hasil nyata. Saat

dokumentasi, dashboard mencatat 9 foto dipindai, 8 wajah terdeteksi, dan 3 di antaranya diklasifikasikan memakai masker (37,5%). Angka ini adalah keluaran sistem, bukan perbandingan dengan label sebenarnya. Waktu proses pada contoh Gambar 4 sekitar 262 ms per foto.

Skenario pengujian	Perilaku sistem
Foto berisi wajah	Kotak, label, dan skor tampil; riwayat dan statistik diperbarui.
Foto tanpa wajah	Pesan Wajah tidak ditemukan disertai saran foto yang lebih baik.
Berkas bukan gambar	Pesan format tidak didukung, tidak ada data yang dikirim.
Gambar lebih dari 5 MB	Pesan batas ukuran 5 MB.
Server tidak aktif	Pesan tidak dapat terhubung ke server beserta perintah menjalankan backend.
Izin kamera ditolak	Pesan akses kamera ditolak dan saran memakai Pilih foto.
Belum ada data	Empty state pada panel Hasil analisis dan Riwayat pemindaian.

Keterbatasan

- Masker berwarna mirip kulit atau pencahayaan ekstrem dapat membuat klasifikasi keliru.
- Haar Cascade dirancang untuk wajah menghadap depan; wajah miring, tertutup sebagian, atau sangat kecil dapat tidak terdeteksi.
- Skor keyakinan belum terkalibrasi. Usia dan jenis kelamin belum tampil karena modelnya belum dipasang.

Rencana pengembangan: memasang model MobileNetV2 hasil transfer learning, menyiapkan dataset uji berlabel, lalu mengevaluasi dengan confusion matrix, accuracy, precision, dan recall.

5. Tautan dan Cara Menjalankan

Repository GitHub: https://github.com/ditormdhn54-sy/maskguard

- Pasang Python 3.10 atau lebih baru.
- Windows: jalankan run.bat di folder proyek. Manual: `pip install -r requirements.txt`, lalu `python -m uvicorn backend.main:app --port 8000`.
- Buka http://127.0.0.1:8000 di browser.